Programming Technology Documentation

Bora NAS

1. Description of the Exercise

3. Hunting

Hunting is a two-player game, played on a board consists of n x n fields, where the first player (call him fugitive) tries to run away, while the second player (the hunter) tries to capture him/her. Initially, the character of the fugitive is at the center of the board, while the hunter has four characters (one at each corner). The players take turns moving their character (hunter can choose from 4) 1 step on the board (they cannot step on each others character). The objective of the hunter is to surround the fugitive in at most 4n steps, so it won't be able to move. Implement this game, and let the board size be selectable (3x3, 5x5, 7x7 → turns are 12, 20, 28). The game should recognize if it is ended, and it has to show the name of the winner in a message box (if the game is not ended with draw), and automatically begin a new game.

2.Uml Diagram



3. Short Description of Each Method

• Field Class (Model)
Field()
Constructor that initializes the tile object with the default state (EMPTY).
getType()
Returns the current field type (EMPTY, FUGITIVE, or HUNTER).
setType(Type type)
Sets the field state to the specified enum type.
isEmpty()
Returns true if the field type is currently EMPTY, false otherwise.

• HuntingBoard Class (Model)
HuntingBoard(int boardSize)
Constructor that initializes the grid matrix for the selected board dimension ($n \times n$) and sets the turn limit ($4n$).
initPositions()
Resets the grid, places the Fugitive at the center $(n/2, n/2)$, places the 4 Hunters at the corners, and resets turn counters.
handleTileClick(int r, int c)
Processes user clicks based on the current player's turn. Handles Fugitive movement or Hunter selection and subsequent movement.
isValidMove(Point from, Point to)
Validates whether a move is 1 step in any direction (horizontal, vertical, or diagonal) and targets an empty tile.
isOver()
Checks if the game has ended (Fugitive trapped or turn limit reached).
getWinnerMessage()
Returns a formatted message string declaring the winner of the game (Hunter or Fugitive).
hasLegalMoves(Point p)
Evaluates whether a given point (the Fugitive) has at least one valid empty adjacent cell to move into.
getBoardSize()
Returns the current board dimension n.
getMaxTurns()
Returns the total allowed turns ($4n$).
getCurrentTurn()
Returns the current turn number played so far.
isFugitiveTurn()
Returns true if it is currently the Fugitive's turn, false if it is the Hunter's turn.
getField(int r, int c)
Returns the Field instance at grid coordinates (r, c).
getSelectedHunter()
Returns the coordinates of the currently highlighted Hunter, or null if no Hunter is selected.

• **BoardGUI Class**

BoardGUI(int boardSize)

Instantiates the HuntingBoard model, constructs the grid of JButton objects, attaches event listeners, and sets up status labels.

refresh()

Updates the button text labels ('F', 'H', or empty), background colors (Green for Fugitive, Red for Hunter, Orange for selected Hunter, Light Gray for empty), and turn counters on the GUI.

checkGameOver()

Checks if the game is over; if ended, displays a message dialog (JOptionPane) announcing the winner and automatically restarts a new game.

getBoardPanel()

Returns the JPanel containing the grid buttons.

getStatusLabel()

Returns the JLabel displaying game status information.

ButtonListener Class (Inner Controller Class)

ButtonListener(int x, int y)

Constructor that stores the row (x) and column (y) coordinates of the attached grid button.

actionPerformed(ActionEvent e)

Event handler triggered on button click; delegates move processing to handleTileClick(), refreshes the UI, and checks for game completion.

• **HuntingGameGUI Class**

HuntingGameGUI()

Constructor that builds the primary JFrame, adds menu items (JMenuBar) for board size selection (3×3, 5×5, 7×7), and mounts the active BoardGUI.

sizeMenuItem.addActionListener(ActionListener)

Listens for board size menu selections, removes the current BoardGUI panel, initializes a new BoardGUI with the selected size, and calls frame.pack() to automatically fit the window size to the board.

• **Main Class**

main(String[] args)

Entry point of the application; safely launches HuntingGameGUI on the Event Dispatch Thread using SwingUtilities.invokeLater().

4. Connections between the events and event handlers.

Event Source (GUI Element)	User Action (Event)	Event Handler Method	Main Action / Flow
Grid Cell Button (JButton)	Clicking a board cell	ButtonListener.actionPerformed()	Passes cell coordinates to board.handleTileClick(), updates tile colors, and checks for game over.
Size Menu Item (JMenuItem)	Selecting 3×3, 5×5, or 7×7	ActionListener.actionPerformed()	Removes the old board, creates a new BoardGUI with the selected size, and resizes the window using frame.pack().
Exit Menu Item (JMenuItem)	Clicking "Exit"	ActionListener.actionPerformed()	Closes the application via System.exit(0).
Game Over Event	Game reaches win condition	BoardGUI.checkGameOver()	Displays a message box (JOptionPanel) showing the winner, then automatically resets the board to start a new game.